pandyaHomeLab

Stage 1 · Conceptual Design

A secured, cloud-architecture-savvy AI showcase platform

Architect & Author · Archit Pandya

pandyahomelab.com · April 2026 · v1.0

What is pandyaHomeLab

A personally architected, on-premises AI/ML platform — built to demonstrate end-to-end machine learning and deep learning capabilities, mirrored to AWS for hybrid cloud parity.

Mission

- Serve as a credible portfolio anchor for an AI career transition.
- Operate as a continuously evolving learning environment.
- Demonstrate production-grade architecture patterns at non-production scale.

Secured

Realistic threat model. TLS everywhere. Network isolation by tier.

Cloud-architecture-savvy

VPCs, subnets, multi-AZ patterns. AWS-equivalent on Synology.

Non-commercial

Zero revenue at stake. Permission to do it the right way.

Learning-deliberate

Every choice is documented and defensible.

Mental model · AWS ↔ Synology

Each Synology component has an AWS equivalent. Same architectural problem, different vocabulary.

AWS		Synology
VPC · 10.0.0.0/16	≡	Router LAN · 192.168.1.0/24
Subnets (public, private)	≡	Docker networks · bridge, macvlan
Security groups + NACLs	≡	DSM Firewall + router rules
Internet Gateway + Elastic IP	≡	Port forward + Dynamic DNS
Route 53 + ACM	≡	Hostinger DNS + Let's Encrypt
ALB / API Gateway	≡	Nginx reverse proxy
ECS / EKS / EC2	≡	Container Manager · Docker

Network architecture · six logical networks

Networks isolate by trust boundary, not by category. Same layout on both platforms.

proxy-network

Reverse proxy · TLS termination · Public ingress

data-network

Postgres · MinIO · Redis · Shared persistence

mlops-network

MLflow · Prometheus · Grafana · Cross-domain

Shared services tier

ml-network

Classical ML · classification, regression, clustering

dl-network

Deep learning · CNN, transformers · GPU workloads

nlp-network

Language tasks · text classification, NER

agentic-network

Agents · RAG · prompt engineering · chatbots

Domain tier

AWS implementation · multi-AZ

One VPC, six logical networks, twelve subnets. AZ-A and AZ-B mirror for HA.

VPC · 10.1.0.0/16

us-east-1a		us-east-1b	
proxy-network	10.1.0.0/24	proxy-network	10.1.10.0/24
data-network	10.1.1.0/24	data-network	10.1.11.0/24
mlops-network	10.1.2.0/24	mlops-network	10.1.12.0/24
ml-network	10.1.3.0/24	ml-network	10.1.13.0/24
dl-network	10.1.4.0/24	dl-network	10.1.14.0/24
nlp-network	10.1.5.0/24	nlp-network	10.1.15.0/24
agentic-network	10.1.6.0/24	agentic-network	10.1.16.0/24

Synology implementation · Docker networks

Same six logical networks, implemented as Docker bridges in RFC 1918 space.

Network	CIDR	Tier	Purpose
proxy-network	172.18.0.0/24	Shared	Reverse proxy · publishes ports 80/443 to host
data-network	172.18.1.0/24	Shared	Postgres, MinIO, Redis · no internet egress
mlops-network	172.18.2.0/24	Shared	MLflow, Prometheus, Grafana
ml-network	172.19.0.0/24	ML	Classification, regression, clustering APIs
dl-network	172.20.0.0/24	DL	GPU workloads · CNN, transformers, vision
nlp-network	172.21.0.0/24	NLP	Text classification, NER, language tasks
agentic-network	172.22.0.0/24	Agentic	Agents, RAG, prompt patterns · vector DB

Traffic flow · click to container

Six steps from a user clicking a link to landing on a running model API.

Step	Component	Action	Detail
1	**Browser**	User clicks link	*https://pandyahomelab.com/ml/iris-knn*
2	**Hostinger DNS**	Resolves the hostname	*A record → 100.35.208.212*
3	**Fios router**	Port forwards on public IP	*443/TCP → 192.168.1.152:8443*
4	**Synology + Nginx**	Terminates TLS, reads URL	*Lets Encrypt cert · pandyahomelab.com*
5	**Nginx routing**	Path → upstream container	*/ml/iris-knn → ml-iris-knn:8001*
6	**Container**	FastAPI handles the request	*Inference · returns JSON*

URL hierarchy · progressive disclosure

Each level narrows user intent. Levels 1–3 are static HTML. Level 4 is a running container.

L1

What is this site?

`pandyahomelab.com/`

Landing page · static index.html

L2

What field am I in?

`/ml/` `/dl/` `/nlp/` `/agentic/`

Domain section · static HTML

L3

What problem am I solving?

`/ml/classification/`

Technique family · static HTML

L4

What demo am I about to use?

`/ml/classification/iris-knn`

Running container · FastAPI

Naming convention · technique vs algorithm

Level 3 is the problem family. Level 4 is dataset-algorithm. One rule, no debate.

The principle

Level 1: site identity
Level 2: AI domain (ml, dl, nlp, agentic)
Level 3: technique family from textbook taxonomy
classification, regression, clustering ...
Level 4: dataset-algorithm
iris-knn, titanic-randomforest ...

Why dataset first?
Datasets are memorable. Algorithms are technical detail. Lead with what sticks.

Example · ML domain

```
/ml/
    /ml/classification/
        /ml/classification/iris-knn
        /ml/classification/iris-svm
        /ml/classification/titanic-randomforest
    /ml/regression/
        /ml/regression/housing-linear
        /ml/regression/housing-xgboost
    /ml/clustering/
        /ml/clustering/customers-kmeans
```

Project scope · four AI domains

Each domain hosts technique families and dataset-algorithm demos.

Machine Learning
/ml/

Classical statistical methods

- classification
- regression
- clustering

Deep Learning
/dl/

Neural networks · GPU

- image classification
- object detection
- segmentation

NLP
/nlp/

Language understanding

- text classification
- named entity recognition
- summarization

Agentic AI
/age/

LLM-based systems

- prompt engineering
- RAG
- chatbot assistants

What Stage 1 deliberately excludes

Naming what's NOT in this stage prevents scope creep and keeps the deliberable shippable.

×	**Container catalog & versions**	*Specific MLflow version, Postgres extensions, Jupyter image — all Stage 2.*
×	**docker-compose.yml & Nginx config**	*Actual IaC files come after the conceptual design is locked.*
×	**First demo end-to-end**	*One model, one container, fully wired — Stage 3.*
×	**Terraform & AWS deployment**	*Cloud sync and IaC promotion patterns — Stage 4.*
×	**CI/CD pipelines**	*GitHub Actions, image builds, automated deploys — Stage 4.*
×	**TLS automation & monitoring**	*Let's Encrypt automation, Prometheus alerts, dashboards — Stage 5.*
×	**Threat model document**	*Two-page formal threat model, signed off — early Stage 2 or 5.*

Roadmap · five stages

Stage 1 is locked. Each subsequent stage builds on what came before.

1	**Conceptual design**	Network base, traffic flow, URL hierarchy, scope	LOCKED
2	**Services & implementation**	Folder structure, containers, Nginx, docker-compose	
3	**First demo end-to-end**	One model on one container, fully wired	
4	**CI/CD & AWS sync**	Terraform, GitHub Actions, environment promotion	
5	**Hardening & observability**	TLS automation, monitoring, alerts, threat model	

Stage 1 · locked

The base is sound.

Next: Stage 2 · Services & Implementation